File	Lines	Code	CVF	Severity	Category	Description	Recommendation	Client comment	Status
NumericArrayLib.sol	3	`3pragma solidity ^0.8.17;`	1	Minor	Procedural	Consider specifying as "^0.8.0" unless there is something regarding this particular version.		We use custom errors (supported only in >0.8.4 and we wanted to avoid bugs in assembly introduced in 0.8.13 and 0.8.14)	Info
NumericArrayLib.sol	6–7	`6 // This function sort array in memory using bubble sort algorithm,` `7 // which performs even better than quick sort for small arrays`	2	Minor	Documentation	It is unclear what function this comment refers to.	Put the comment right before the corresponding function.		Fixed

NumericArrayLib.sol	33, 37, 41	`33 if (arr.length == 2) {`
		`37 if (arr.length == 1) {`
		`41 if (arr.length == 0) {`
			3	Minor	Readability	Should be "else if" for readability.		It would generate a bigger block oof code. IMO the early return pattern, which is used here, makes the code a bit more readable	Info

NumericArrayLib.sol	46	`46 sort(arr);`	4	Moderate	Suboptimal	Sorting an array just to find its median is suboptimal.	Use the quickselect algorithm.		Info
NumericArrayLib.sol	56	`56 function sort(uint256[] memory arr) internal pure {`	5	Minor	Documentation	Despite the comment above, this function doesn't implement the bubble sort algorithm, as bubble sort always compare adjacent element.	Either fix the comment or actually implement the bubble sort algorithm.		Fixed
NumericArrayLib.sol	60	`60 let endPtr := add(valuesPtr, mul(arrLength, UINT256_VALUE_BS))`	6	Minor	Suboptimal	Left shift would be more efficient than multiplication.			Fixed
NumericArrayLib.sol	62	`62 let arrIPtr := valuesPtr`	7	Minor	Suboptimal	Starting from the first array element here is inefficient, as there are no indexes for "J" to be iterated.	Start from the second element.		Fixed
IOracleAggregator.sol	3	`3pragma solidity ^0.8.17;`	8	Minor	Procedural	Consider specifying as "^0.8.0" unless there is something regarding this particular version.		We use custom errors (supported only in >0.8.4 and we wanted to avoid bugs in assembly introduced in 0.8.13 and 0.8.14)	Info

IOracleAggregator.sol	11–13	`11 function chronicle() external view returns (address chronicle);`
		`12 function chainlink() external view returns (address chainlink);`
		`13 function redstone() external view returns (address redstone);`
			9	Minor	Bad datatype	The return type for these functions should be more specific.			Fixed

OracleAggregator.sol	3	`3pragma solidity ^0.8.17;`	10	Minor	Procedural	Consider specifying as "^0.8.0" unless there is something regarding this particular version.		We use custom errors (supported only in >0.8.4 and we wanted to avoid bugs in assembly introduced in 0.8.13 and 0.8.14)	Info

OracleAggregator.sol	35–37	`35 function chronicle() public view virtual returns(address);`
		`36 function chainlink() public view virtual returns(address);`
		`37 function redstone() public view virtual returns(address);`
			11	Minor	Bad datatype	The return type for these functions should be more specific.			Fixed

OracleAggregator.sol	47–48	`47 uint256 index = 0;` `48 goodOraclesCount = (okChronicle ? 1 : 0) + (okChainlink ? 1 : 0) + (okRedstone ? 1 : 0);`	12	Moderate	Suboptimal	The final value of "index" equals to the number of good oracles. No need to calculate this values separately.	Don't calculate "goodOraclesCount" separately. Just use the final "index" value instead.	There is a need, as we need to know the size of the array to allocate memory for it. It's done before the index value is equal to goodOraclesCount	Info
OracleAggregator.sol	48	`48 goodOraclesCount = (okChronicle ? 1 : 0) + (okChainlink ? 1 : 0) + (okRedstone ? 1 : 0);`	13	Minor	Suboptimal	This could be optimized by calculating the sum of the flags via an assembly block.			Fixed
OracleAggregator.sol	64	`64 function _safeLatestRoundData(address priceFeed) internal view returns (bool success, uint256 val) {`	14	Minor	Bad datatype	The argument type should be more specific.			Fixed
OracleAggregator.sol	79	`79 isStale = updatedAt + getAgeThreshold() < block.timestamp;`	15	Major	Overflow/Underflow	Overflow is possible here.	Rewrite as: updatedAt < block.timestamp - getAgeThreshold()		Fixed

OracleAggregator.sol	86, 89, 119, 122	`86 return (true, uint256(answer));`
		`89 return (false, 0);`
		`119 return (true, value);`
		`122 return (false, 0);`
			16	Minor	Readability	Should be "else return" for readability.		We prefer using early return pattern here	Info

OracleAggregator.sol	94–95	`94 * @return success If the call succeeded and is not stale` `95 * @return val The oracle value`	17	Minor	Bad naming	The returned values actually don't have names.	Give names to the returned values.		Fixed
OracleAggregator.sol	110	`110 isStale = age + getAgeThreshold() < block.timestamp;`	18	Major	Overflow/Underflow	Overflow is possible here.	Rewrite as: age < block.timestamp - getAgeThreshold()		Fixed
ETH_Oracle_Aggregated_Redstone_Chainlink_Chronicle.sol	3	`3pragma solidity ^0.8.17;`	19	Minor	Procedural	Consider specifying as "^0.8.0" unless there is something regarding this particular version.		We use custom errors (supported only in >0.8.4 and we wanted to avoid bugs in assembly introduced in 0.8.13 and 0.8.14)	Info

ETH_Oracle_Aggregated_Redstone_Chainlink_Chronicle.sol	15, 19, 24, 28	`15 return 0x46ef0071b1E2fF6B42d36e5A177EA43Ae5917f4E;`
		`19 return 0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419;`
		`24 return 0xdDb6F90ffb4d3257dd666b69178e5B3c5Bf41136;`
		`28 if (consumer != 0x8105f69D9C41644c6A0803fDA7D03Aa70996cFD9) {`
			20	Moderate	Suboptimal	Hardcoding mainnet addresses is a bad practice, as it makes it harder testing the code.	Use immutable variables instead and pass addresses as constructor arguments.	This contract is intented to be a configuration contract. Using storage for these variable will increase gas cost significantly	Info

ETH_Oracle_Aggregated_Redstone_Chainlink_Chronicle.sol	34	`34 return 25 days;`	21	Minor	Bad naming	This value should be a named constant.		This is a configuration contract. We configure values here directly in the overriden functions. Adding such a constant will increase the code size without increasing readability of the code	Info